Source: http://localhost:3333/FIXES-REPORT-SOURCE.md

Langfuse Docs — Fixes Report

Branch: `to-fumadocs` **Date:** 2026-03-13 **Stack:** Next.js 16 App Router + Fumadocs (migrated from Nextra Pages Router)

Summary

This report documents all fixes applied during the Nextra → Fumadocs migration. Fixes are grouped by session. All items marked ✅ are resolved and verified against a production build (312 static pages, 0 errors).

Session 1 — Technical Issues (Branch: `technical-issues`)

Fix 1 — metadataBase in root layout

Problem: Relative OG/canonical URLs were generated without a base domain, producing malformed meta tags on deployed environments.

Fix: Added `metadataBase: new URL("https://langfuse.com")` to the root layout export in `app/layout.tsx`.

Status: ✅ Resolved

Fix 2 — og:url, canonical, seoTitle in route metadata

Problem: Route-level `generateMetadata` functions in docs, guides, integrations, library, self-hosting, section, and wide pages were missing `alternates.canonical`, `openGraph.url`, and `seoTitle` support.

Fix: Updated all route `generateMetadata` functions to use `buildPageUrl`, set `alternates: { canonical }`, `openGraph: { url }`, and resolve `seoTitle ?? title` for the HTML `<title>` element.

Status: ✅ Resolved

Fix 3 — buildPageUrl helper

Problem: No shared utility existed for constructing absolute canonical URLs, causing inconsistency across routes.

Fix: Added `buildPageUrl(path: string)` to `lib/og-url.ts` returning `https://langfuse.com` + the given path.

Status: ✅ Resolved

Fix 4 — seoTitle & ogVideo in frontmatter schemas

Problem: `seoTitle` and `ogVideo` frontmatter fields were not declared in `source.config.ts` schemas, causing them to be stripped from page data.

Fix: Added `seoTitle` (optional string) and `ogVideo` (optional string) to the relevant collection schemas in `source.config.ts`.

Status: ✅ Resolved

Fix 5 — ogVideo absolute URL handling

Problem: In `app/[section]/[[...slug]]/page.tsx`, relative `ogVideo` values were not prefixed with the site base URL, and absolute URLs could get double-prefixed.

Fix: Added a guard: if `ogVideo` starts with `http`, use it as-is; otherwise prefix with `buildPageUrl`.

Status: ✅ Resolved

Fix 6 — seoTitle on four overview pages

Problem: Four key overview pages had `title: Overview`, causing the browser tab and search results to show the generic "Overview - Langfuse" instead of descriptive titles.

Fix: Added `seoTitle` frontmatter to:

- `content/docs/observability/overview.mdx`
- `content/docs/prompt-management/overview.mdx`
- `content/docs/evaluation/overview.mdx`
- `content/docs/api-and-data-platform/overview.mdx`

Status: ✅ Resolved

Fix 7 — H4 → H3 heading (prompt-management get-started)

Problem: A heading in `content/docs/prompt-management/get-started.mdx` used H4 after H2, skipping H3, causing an accessibility violation (heading level skip).

Fix: Changed the offending `####` to `###`.

Status: ✅ Resolved

Fix 8 — Feedback widget on FAQ/section pages

Problem: FAQ and other section pages rendered through the `[section]` route lacked the `DocsFeedback` and `DocsSupport` widgets at the bottom of each page.

Fix: Section pages that use prose layout now render the full `DocBodyChrome` (see Fix 13), which includes both widgets.

Status: ✅ Resolved

Fix 9 — FAQ index sidebar label

Problem: The FAQ index page appeared in the left sidebar with no label or an incorrect label.

Fix: `content/faq/index.mdx` now has `title: Overview` and `seoTitle: "Langfuse FAQ – Most Common Questions"`.

Status: ✅ Resolved

Fix 10 — md-to-pdf runtime and timeout

Problem: The `app/api/md-to-pdf/route.ts` was running on the Edge runtime, which is incompatible with Puppeteer/Chromium. It also had an insufficient 10 s timeout.

Fix: Added `export const runtime = "nodejs"` and `export const maxDuration = 60`.

Status: ✅ Resolved

Fix 11 — Untrack .source/source.config.mjs

Problem: The generated `.source/` directory (Fumadocs internal cache) was being tracked by git.

Fix: Added `.source/` to `.gitignore` and removed the tracked file.

Status: ✅ Resolved

Fix 12 — README updated

Problem: The README still referenced Nextra and the old `pages/` directory structure.

Fix: Updated README to reflect Fumadocs + App Router, correct content paths (`content/<section>/`), and accurate dev/build instructions.

Status: ✅ Resolved

Session 2 — Multiple Content Registries & Nextra Shim RSC Fixes

Fix 13 — Eliminated parallel client-side MDX loader registries

Problem (High Priority): The codebase maintained two redundant client-side MDX loader systems:

1. `lib/section-loaders.generated.ts` — generated at prebuild by `scripts/generate-section-loaders.js`; contained dynamic `import()` factories for every MDX file in every section
2. `app/docs/[[...slug]]/doc-loaders.client.ts` — the same pattern for docs specifically
3. `app/[section]/SectionDocBodyClient.tsx` — a "use client" component that called these loaders
4. `components/SectionDocBodyClientWithDocsBody.tsx` — a similar "use client" wrapper for self-hosting/guides/integrations/library

This created O(N) client bundle weight (one dynamic import per MDX file), slow initial load, and maintenance overhead from the generated file.

Fix: Replaced the entire system with server-side MDX rendering:

- **`components/DocBodyChrome.tsx`** (new) — thin `"use client"` wrapper that receives pre-rendered MDX as `children` and adds `DocsBody`, `CopyMarkdownButton`, `DocsFeedback`, `DocsSupport`, and `NotebookBanner` chrome. All interactive elements are client-side; the document content itself is server-rendered.
- **All route pages** (`app/docs`, `app/guides`, `app/integrations`, `app/self-hosting`, `app/library`, `app/[section]`) now render `page.data.body` on the server directly:

```
const MDX = page.data.body as ComponentType<{ components?: ... }>;
return (
```

```
  <DocBodyChrome>
    <MDX components={getMDXComponents()} />
  </DocBodyChrome>
);
```

- **`app/(wide)/WideSectionPage.tsx`** similarly renders MDX on the server with no chrome wrapper.

Deleted files:

- `app/[section]/SectionDocBodyClient.tsx`
- `components/SectionDocBodyClientWithDocsBody.tsx`
- `app/docs/[[...slug]]/DocBodyClient.tsx`
- `app/docs/[[...slug]]/doc-loaders.client.ts`
- `lib/section-loaders.generated.ts`
- `scripts/generate-section-loaders.js`

`package.json` prebuild: Removed the `generate-section-loaders.js` invocation.

Status: ✅ Resolved

Fix 14 — Nextra shim: Cards.Card, Tabs.Tab, FileTree.File/.Folder not accessible in RSC

Problem (High Priority): Moving MDX rendering to the server (Fix 13) surfaced a fundamental RSC issue with the nextra-shim components.

Root cause: In React Server Components, imports from `"use client"` modules become opaque *client references* — proxy objects that React serialises into the RSC payload. Property access on a client reference returns `undefined`:

```
// In RSC context:
import { Tabs } from "lib/nextra-shim/components"; // "use client" module
Tabs.Tab  // → undefined (property on client reference)
```

The original `lib/nextra-shim/components.tsx` had `"use client"` at the top. When MDX files imported directly (`import { Tabs } from "nextra/components"`) or when the compiled MDX accessed `_components.Tabs.Tab`, the result was `undefined` → build error or invisible tab content.

Additionally, the `Tabs` component injected `value` props into its `Tab` children via `React.Children.map` with a `child.type !== Tab` check. When running on the server, this check could fail because the client reference resolved on the server has no meaningful type for comparison.

Fix — three-layer shim split:

`lib/nextra-shim/cards.tsx` (new, no `"use client"`):

- `Cards`, `Card`, `Callout`, `Steps`, `Playground` — all hook-free, fully server-safe
- `Cards.Card = CardComponent` is set on a real function → accessible in RSC

`lib/nextra-shim/tabs-client.tsx` (new, `"use client"`):

- `TabsClient` — the hook-based tab-switching logic (`useState`, `useEffect`, `localStorage`)
- `FileTreeFolder` — uses `useState` for open/close toggle

`lib/nextra-shim/tabs.tsx` (rewritten, no `"use client"`):

- `Tab` — server-safe, renders `FumadocsTabsContent`
- `Tabs` — server-safe wrapper: computes `values = items.map(toValue)`, injects values positionally into children via `React.cloneElement`, then renders `<TabsClient items values> {injectedChildren}</TabsClient>`
- `FileTreeFile` — server-safe
- `FileTree` — server-safe; `.File = FileTreeFile` and `.Folder = FileTreeFolder` are set as plain properties (accessible in RSC since `FileTree` is a real function)

`lib/nextra-shim/components.tsx` (updated, no `"use client"`):

- Re-exports from both `./cards` and `./tabs` — no client boundary here

`mdx-components.tsx` (updated):

- Added compound-key entries: `"Tabs.Tab": Tab`, `"Cards.Card": Card`, `"FileTree.File": FileTreeFile`, `"FileTree.Folder": FileTreeFolder`

MDX content files — removed all `import { Tabs, Cards, Callout, FileTree } from "nextra/components"` lines from:

- `components-mdx/integration-learn-more.mdx`
- `components-mdx/integration-learn-more-js.mdx`
- `components-mdx/self-host-features.mdx`
- `components-mdx/get-started/next-steps.mdx`
- 15 files in `content/` (self-hosting, docs, changelog, blog, integrations, marketing)

These files now rely on components passed via `getMDXComponents()` rather than direct module imports, which avoids the RSC client reference limitation.

Status: ✅ Resolved — tabs switch correctly, 312 static pages build with 0 errors

---

Build Verification

```
✓ Compiled successfully in 38.5s
✓ TypeScript: 0 errors
✓ Generating static pages using 9 workers (312/312)
```

All routes (docs, guides, integrations, self-hosting, library, section, wide) return HTTP 200 in development and generate correctly in production.

Remaining Items (Not Yet Fixed)

Item	Priority	Notes
Bundle size regression	Medium	Audit JS bundle after Fix 13; server-side MDX should help significantly
Sitemap canonical/noindex	Medium	Some redirected paths may appear in sitemap
Inkeep 403 on preview	Low	Preview environment API key scoping issue
DocsLayout duplication	Low	Layout wrapping docs and some section pages may be duplicated